

AIRFLOW SUMMIT 2026

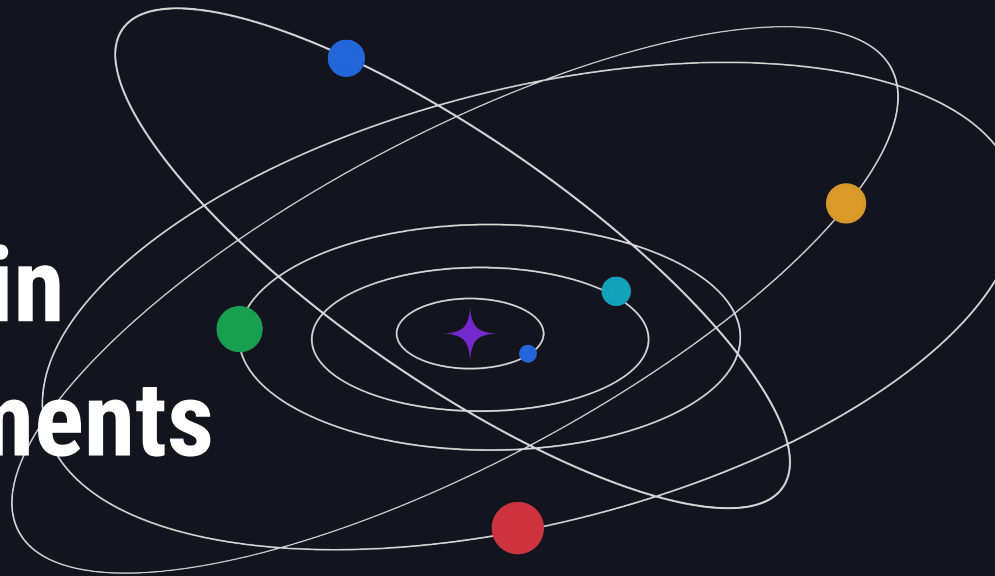
Optimising Airflow in Real-World Deployments

A framework for finding what is actually slow
and knowing what to do about it

Vara Prasad Regani, Senior Airflow Reliability Engineer, Astronomer

Pankaj Koti, Senior Software Engineer, Astronomer

ASTRONOMER



Performance problems do not announce themselves

They show up as drift. Small signals that are easy to dismiss.

"My DAGs are slow"

"Tasks stay queued forever"

"The UI is laggy"

"The scheduler keeps restarting"

"Airflow 3 made things worse"

Every one of these has a **different root cause**. And without checking, you cannot tell which one you are dealing with.

START WITH THE SYMPTOM

Start with the symptom, not the config.

Slow to appear?

→ DAG parsing

Slow to start?

→ Scheduling and queue time

Slow after it starts?

→ Task runtime and workers

UI or API slow?

→ Metadata DB and API server

Worse after an upgrade?

→ Version-specific effects

The first question is **"where is the time being spent?"** not **"what should we tune?"**

Use the lightest tool that answers the question

Airflow metrics and logs

Use first. Which component is unhealthy?

Database inspection

When several components are slow together.

py-spy

When a process burns CPU. What code is running?

Memray

When memory grows or a process gets OOM-killed.

Application profiling

When the task itself is slow after it starts.

Do not profile everything. Pick the tool based on the failing signal.

Each Airflow component fails differently

- **DAG processor**

parse time, import errors, restarts

- **Scheduler**

loop duration, queued count, latency

- **Workers**

runtime, memory, OOM kills, timeouts

- **Triggerer**

deferred count, capacity, heartbeat

- **Metadata DB**

table size, pool wait, slow plans, bloat

- **API server / UI**

latency, memory growth, pool wait

What is failing? That decides what to inspect.

DAGs appear late after deploy

→ parse stats, per-file parse time, top-level code

Tasks wait before running

→ queue timestamps, pools, parallelism, loop duration

Tasks run slowly

→ task logs, duration history, py-spy, Memray

Workers restart or disappear

→ OOMKilled events, worker memory, heartbeat timeout

Triggerer not resuming tasks

→ triggerer logs, deferred count, py-spy on event loop

UI or API is slow

→ API server memory, DB query plans, PgBouncer wait

Upgrade regresses

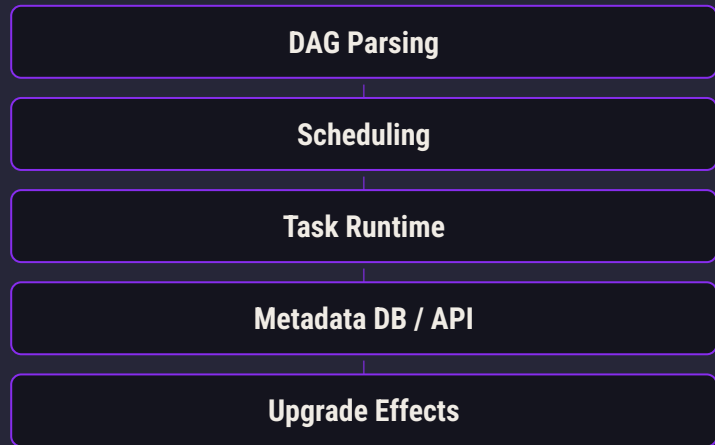
→ metadata size, XCom size, Ruff AIR rules, providers

THE FRAMEWORK

Check, identify, act

- 1 **Classify the symptom**
- 2 **Pick the layer**
- 3 **Measure the bottleneck**
- 4 **Change one thing**
- 5 **Verify the signal moved**

THE DEFAULT PATH

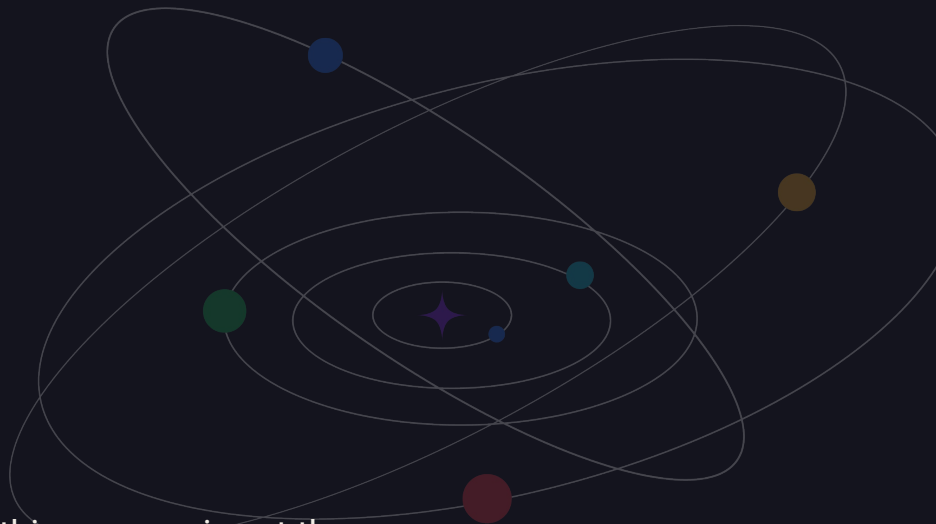


LAYER 1

DAG Parsing



The DAG processor re-parses every file on a loop. Anything expensive at the top level is not setup code. It is parsing workload, repeated on every cycle.



What to check

One log line, already in your logs. No install needed.

```
grep 'DAG File Processing Stats' scheduler.log
```

```
[2025-04-04T22:01:03.177+0000] {manager.py:997} INFO -
```

```
=====
```

```
DAG File Processing Stats
```

File Path	PID	Runtime	# DAGs	# Errors	Last Runtime	Last Run	Last # of DB Queries
/usr/local/airflow/dags/ingestion/generic/api.py			33	0	9.62s	2025-04-04T22:00:44	855
/usr/local/airflow/dags/ingestion/generic/database.py			25	0	4.58s	2025-04-04T22:00:53	650
/usr/local/airflow/dags/ingestion/generic/setup.py			3	0	3.82s	2025-04-04T22:00:40	115
/usr/local/airflow/dags/ingestion/generic/dbt.py			2	0	2.85s	2025-04-04T22:00:47	207
/usr/local/airflow/dags/competition/gns_ingestion.py			3	0	1.37s	2025-04-04T22:00:43	36
/usr/local/airflow/dags/ingestion/generic/tests.py			1	0	1.13s	2025-04-04T22:00:45	45
/usr/local/airflow/dags/ingestion/generic/source_metadata_alerts.py				1	0 1.08s	2025-04-04T22:00:59	
/usr/local/airflow/dags/monitoring/salesforce_sync_alert.py				1	0 1.04s	2025-04-04T22:00:44	45
/usr/local/airflow/dags/competition/statsperform_historical_load.py				1	0 0.49s	2025-04-04T22:00:43	
/usr/local/airflow/dags/competition/simulation.py			1	0	0.40s	2025-04-04T22:00:46	12

04/05/25
00:01:03 AM

What slow parsing actually means

1

The DAG Processor falls behind

DAGs look stale. New deploys take minutes.

2

The Processor's heartbeats slip

Kubernetes kills it. It restarts and re-parses.

3

The scheduler sees incomplete state

"DAG not found" until the next good parse.

ALMOST ALWAYS ONE OF FOUR ROOT CAUSES

top-level Variable.get / Connection · API or DB calls outside a task · heavy imports · dynamic DAGs re-discovering inputs

What we found on real deployments

214s to parse one file

214s to parse one file, Four files over 2 min each. Processor restarted every 3–5 min.

203 DAGs in one file

203 DAGs in one file If that file fails to parse, all 203 DAGs disappear from the UI at once.

linear cost per remote job

A DAG file called a live API at parse time. Parse cost grew with every new remote job.

~40% faster with orjson

Cosmos JSON parse on a 30MB+ manifest was the bottleneck.

What to do about slow parsing

- Move external calls into tasks, not module scope
- Split large generator files
- Cache or pre-compute discovery at deploy time
- Tune `min_file_process_interval` and `parsing_processes` before resizing
- If you use Cosmos, measure caching and loading modes before switching

LAYER 2

Scheduling and Task Latency

Tasks can be slow for reasons that have nothing to do with the task code. Queue time, concurrency limits, and pool slots sit between scheduled and running.

What to check

Measure **queue time** against **run time**. High queue, normal duration means they are waiting, not slow.

CONCURRENCY LIMITS, CHECKED IN ORDER · any one gates throughput even if workers are idle

1. parallelism
2. max_active_tasks_per_dag
3. max_active_runs_per_dag
4. pool slots
5. worker concurrency

What to do about scheduling bottlenecks

- Tasks waiting? The bottleneck is before the worker, check limits and pools
- Scheduler restarting? Raise the health threshold, then fix why the loop is slow
- Workers scale up but throughput is flat? A concurrency cap is the real limit
- Tune from a known baseline: understand each setting before changing it

LAYER 3

Task Runtime and Worker Behaviour



Once a task starts, Airflow is not the only system involved. Prove whether the task is waiting to start or slow after it starts. The fix is different.

Slow after they start? Match the signal to the tool

Rising p95, stable median

→ intermittent contention, either in the worker or an external system

Memory climbs, does not fall

→ Memray: what allocated this memory, and what is still holding it?

CPU high on Python work

→ py-spy: which functions are consuming the most time?

CPU and memory both low

→ the worker is waiting on something outside Airflow

```
py-spy top -p <pid>      # live CPU
py-spy record -p <pid>    # flame graph
```

What we found on real deployments

Worker CPU pegged

A task-supervisor busy-loop from a slow metadata DB, not a heavy task.

Scheduler memory leak

Memray found retained serialized DAG versions accumulating after an upgrade.

Triggerer unresponsive

Custom triggers doing synchronous work inside async code blocked the event loop.

88% CPU throttled

Sidecar processes, not Airflow, consumed the pod's CPU resources.

How much memory does a task really use?

worker_concurrency is usually a guess. **Too high OOM-kills tasks. Too low wastes workers.**

$$\text{worker_concurrency} \approx \frac{\text{worker memory}}{\text{safety}} / (\text{peak task memory} \times)$$

The one number we are missing is **peak memory per task**, measured on a real run, not estimated.

Measure it: Cosmos debug mode

TURN IT ON

```
[cosmos]
enable_debug_mode = True
debug_memory_poll_interval_seconds = 0.5
```

Example run · peak RSS per task

dbt_run_staging	612 MB
dbt_run_marts	1,240 MB
dbt_test	880 MB

HOW IT WORKS

- Polls the task process RSS every 0.5s during dbt execution
- Writes the max to an XCom: cosmos_debug_max_memory_mb

8 GB worker / (1,240 MB × 1.3)

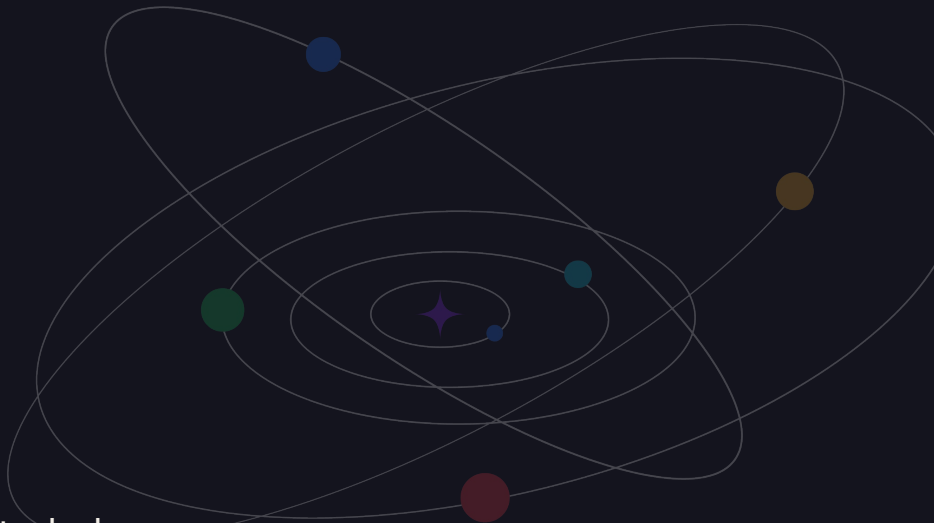
≈ 5 → set worker_concurrency = 4

Cosmos PR #2327 · github.com/astronomer/astronomer-cosmos/pull/2327

LAYER 4

Metadata Database

The scheduler, every worker, the API server, and every task share one database. When it slows, everything slows, gradually enough that nobody notices. If many components are slow at once, jump here first.



Three things, regardless of which database you use.

1. Table & index sizes

task_instance over 10 GB, log table growing unbounded, xcom over a few GB, dead rows accumulating.

2. Unused indexes

Indexes never read by any query are pure write overhead on every INSERT and UPDATE.

3. Slow queries

Use your database's execution plan tool on the queries the scheduler runs every loop.

What we found on real deployments

101 GB

database

The cleanup that was not cleaning.
Deletes silently failed for months.

51 GB

XCom table

XCom table Blocked an entire Airflow 3 migration. Had to roll back and truncate first.

68 GB

index

Would not shrink after deleting 14M rows. Only REINDEX reclaimed the space.

90x from one index · real EXPLAIN ANALYZE

span_status lookup	6,867 ms	→	0.04 ms
updated_at + dag_id	35,692 ms	→	620 ms

"It worked fine for years" is not a guarantee

Long-running environments accumulate metadata until the same query costs **1,000x more**.

Bloat hides below the autovacuum threshold

task_instance	412,726	3.4%
rendered_task..	278,383	7.6%
dag_run	61,514	6.6%
xcom	55,613	3.9%

Autovacuum waits for 50 + 10% of live rows. On millions of rows that threshold is huge.

Treat DB health as a recurring job, not a one-time task.

What to do about metadata DB problems

Run cleanup, then verify sizes actually dropped

Check XCom size before any upgrade

Rebuild indexes after large deletes

Add indexes only after EXPLAIN ANALYZE

Drop unused, zero-scan indexes

Keep PgBouncer sized for real connection demand

AFTER AN UPGRADE

Same framework, version lens

An upgrade does not create most problems. It surfaces state and assumptions that were already there but invisible. Run the same framework, add version-specific checks at each layer.

Three changes cause most post-upgrade issues

1

Execution API

Workers no longer touch the DB directly. The API server needs to be sized for request volume and the data it holds in memory.

2

DAG versioning

Every parse that changes the serialized DAG records a new tracked version.

3

DAG bundles

DAGs are versioned bundles, not a flat folder. Imports behave differently.

These issues can layer: on one deployment, each fix revealed a different problem underneath.

AFTER AN UPGRADE

DAG version inflation: the surprise nobody expects

Scheduler memory grows with no code changes. **Something changes on every parse.**

How to check

```
SELECT dag_id, COUNT(*) AS versions
FROM dag_version
GROUP BY dag_id
ORDER BY versions DESC
LIMIT 20;
```

COMMON CAUSES

- `random.randint()` in task params
- `datetime.now()` in `default_args`
- commit SHAs injected at build time
- live object `repr()` in `op_args`

Catch it early: `dag_version_inflation_check_level` (Airflow 3.2+) and Ruff AIR rules.

AFTER AN UPGRADE

The API server is a new surface to size

New in Airflow 3. On the deployments we investigated, memory growth was the leading signal before OOM kills.

Watch

- API server memory over time, not only CPU
- connection-pool wait in API server logs
- DAG-version count for large DAGs

What Helps

- cap versions: dag_cache_size, dag_cache_ttl
- gunicorn worker_refresh_interval
- scale on memory, not just CPU

Before you upgrade to Airflow 3

- 1 Check metadata DB size, clean tables over 10 GB
- 2 Verify the cleanup DAG actually deletes rows
- 3 Grep DAG code for non-deterministic values
- 4 Run Ruff AIR30 and AIR31 compatibility rules
- 5 Size the API server for memory
- 6 Upgrade to 2.11 first, then 3.1+
- 7 Load-test the migration on a proof-of-concept

Check, identify, act

- 1 **DAG Parsing**
→ top-level code, generators, caching, intervals
- 2 **Scheduling Latency**
→ concurrency limits, pools, loop time
- 3 **Task Runtime**
→ py-spy for CPU, Memray for memory, external systems
- 4 **Metadata DB / API**
→ table size, XCom, slow queries, API memory
- 5 **Upgrade Effects**
→ removed APIs, providers, DAG version growth

A dashboard spike is a lead, not a diagnosis. The fix is only real when the original signal improves.

The 2026 Apache Airflow® Survey is Live!

- Takes just 7 minutes
- Get a free AI Orchestration Fundamentals Certification Exam or DAG Authoring Certification (normally \$150 each)



Thank you

Questions welcome.

Vara Prasad Regani, Senior Airflow Reliability Engineer, Astronomer

Pankaj Koti, Senior Software Engineer, Astronomer

Optimising Airflow in Real-World Deployments · Airflow Summit 2026

ASTRONOMER

